Broken on Download? A Functional Acceptance Gate for Quantized GGUF Models, Evaluated on Every Public LFM3-1.7B Quantization

Vatsal Roy
Independent Researcher
https://github.com/VatsalRoy/quant-gate

***Abstract*—Local deployment of large language models increasingly relies on quantized GGUF files downloaded from public registries, yet no step in the distribution pipeline verifies that a file still performs its tasks. We present QUANT-GATE, a functional acceptance gate that runs a quantized file through a short, exactly scored examination of 20 tool-call/strict-JSON cases and 15 executed HumanEval+ problems, and returns a verdict and a CI-compatible exit code. We applied it to all 45 public GGUF files of LiquidAI's LFM3-1.7B from four uploaders, confirmed 23 verdicts on the full benchmarks (164 code problems, 60 tool cases), and ran a controlled rebuild that varies only the importance matrix (imatrix). Every fully benchmarked file below 4.5 bits per weight was degraded or non-functional, with two public files solving 18 and 42 of 164 problems versus 118 for BF16. Mean KL divergence against BF16 ranked files well (Spearman $\rho = -0.84$) but, even with a cutoff tuned in hindsight, admitted two degraded files and rejected a functional one; two rebuilt files with nearly identical KL (0.258 vs. 0.261) solved 86 vs. 104 problems ($p = 0.004$). In the controlled rebuild, any imatrix raised a 2-bit file from 1 to 51–55 solved problems ($p \approx 0$), while at 3 bits the calibration set determined the outcome. Contrary to our hypothesis, code generation degraded long before tool calling. The fast screen took a median of 4.4 minutes per file versus 31 minutes for the full benchmarks; every screen error lay within one problem of a threshold. The gate also exposed a runtime incompatibility and four defects in its own harness, which we document as practical lessons.**

***Index Terms*—large language models, quantization, GGUF, llama.cpp, model registries, acceptance testing, MLOps, edge AI**

I. Introduction

Running language models on laptops, phones and on-premise servers depends on post-training quantization: storing weights with 2–8 bits instead of 16 so that a model fits in memory and runs fast [4]–[6]. In the llama.cpp ecosystem [2], quantized models are distributed as GGUF files, and a single popular model typically has dozens of community-produced files sharing the same quantization names (`Q4_K_M`, `Q3_K_M`, ...) but built with different recipes. Users choose a file by name and size, and nothing in the registry tells them whether it still works.

A recent registry census, "Broken on Arrival" [1], executed 327 code-capable quantized artifacts and found silently defective files in the official Ollama library, all low-bit quantizations of models with at most four billion parameters. That study focused on code models in one registry. Three questions remained open: whether the same risks apply to a new general-purpose small model distributed through Hugging Face community uploaders; whether cheap proxy metrics such as KL divergence can replace task-based screening; and whether agentic capabilities (tool calling, strict JSON), the headline use case for on-device models, degrade differently from code generation.

This paper studies those questions on LiquidAI's LFM3-1.7B [3], a 1.73B-parameter hybrid model released in October 2026 and promoted for on-device agents. Our contributions are:

1) **A functional acceptance gate.** QUANT-GATE is a command-line tool and GitHub Action that issues a verdict for a GGUF file in minutes, separates file defects from packaging, loading and harness failures through distinct exit codes, and records the file hash and the exact runtime binary used (Section III).
2) **A census of all 45 public files** of LFM3-1.7B from four uploaders, with confirmation of 23 screen verdicts on the full benchmarks and an estimate of the screen's miss and false-alarm rates (Sections IV–V).
3) **A controlled rebuild** with three arms (no imatrix and two calibration sets) at three bit widths, in which every tensor's quantization type is verified identical across arms, isolating the effect of calibration.
4) **Evidence on proxy metrics:** KL divergence is a good ranking signal and a poor gate.
5) **Harness lessons:** four defects in our own pipeline that each produced plausible but wrong numbers, with the safeguards now built in (Section VI).

We tested four hypotheses, fixed before the census: **H1**: files at 4 bits and above pass, and failures cluster at 3 bits or below; **H2**: at the same quantization name, imatrix builds outperform plain builds; **H3**: KL divergence ranks files like task accuracy, but no single KL cutoff catches all functional failures; **H4**: tool calling and strict JSON break before code generation as bits decrease.

II. Background and Related Work

A. Quantization and GGUF

Post-training quantization maps each weight block to a low-precision code plus scale factors. GPTQ [4] and AWQ [5]

showed that protecting the weights most important to activations preserves accuracy at low bit widths, and Dettmers and Zettlemoyer [6] characterized accuracy–size trade-offs across bit widths. llama.cpp [2] implements block-wise "K-quants" (e.g., `Q3_K_M`, where the suffix selects a per-tensor mix of precisions) and lattice-based "I-quants" (e.g., `IQ3_XXS`). Its `llama-imatrix` tool collects an *importance matrix* from activations on calibration text; `llama-quantize` can use it to weight the quantization error per input dimension. Whether a published file used an imatrix is recorded in its metadata (`quantize.imatrix.*` keys) but is rarely stated reliably on model cards.

B. Evaluating quantized artifacts

Patodiya [1] introduced registry-scale execution testing with a 15-task smoke screen confirmed on all 164 HumanEval+ tasks, and released the `quantcheck` tool. Liu and Tsai [7] showed that the transformation from signed full-precision weights to a distributed quantized artifact is usually not independently verifiable. Singh [8] found that aggressively quantized small models keep producing fluent text while losing the ability to define rare words, and argued that validation must be per artifact. Our work follows the acceptance-testing approach of [1], extends it to tool calling and to Hugging Face community uploaders, and adds a direct comparison with KL divergence [12] as a screen.

C. Benchmarks and statistics

HumanEval [9] contains 164 Python function-completion problems; EvalPlus [10] extends each with many additional tests (HumanEval+), which catch near-miss solutions. We use Wilson score intervals [13] for small-sample rates, exact McNemar tests [14] for paired per-problem comparisons, and Bonferroni correction [15] for multiple comparisons. KL divergence is computed on WikiText-2 [11].

III. System Design

A. Overview

QUANT-GATE evaluates one GGUF file per invocation:

1) **Metadata check.** Read the GGUF header without running the model: presence and length of the chat template, imatrix keys, and the histogram of tensor quantization types.
2) **Load.** Start `llama-server` with the file's own chat template. Failure to load yields verdict `load_error` (exit code 2).
3) **Tool-call/JSON screen.** 20 cases, scored exactly.
4) **Code screen.** 15 HumanEval+ problems; generated code is executed by EvalPlus.
5) **KL (optional, reported only).** Mean KL divergence and top-1 agreement against saved BF16 logits.
6) **Decision.** Compare scores with thresholds derived from BF16 (Section III-E); emit a JSON report and an exit code.

Exit code 0 means pass, 1 means the file failed (suspect, degraded, non-functional or packaging), 2 means it would not load, and 3 means the tester itself failed. Separating code 3 prevents harness faults from being reported as file defects, a failure mode we encountered repeatedly (Section VI). With `--fail-fast`, evaluation stops as soon as a file can no longer reach a threshold: if the tool screen is already below threshold the code screen is skipped, and code generation stops once more answers have been truncated than the screen can tolerate.

B. Inference settings

All generation uses greedy decoding (temperature 0, seed 0), one request slot and an 8192-token context. LFM3's chat template enables a reasoning mode by default; we disable it through `chat_template_kwargs` for speed and to avoid confounding capability with verbosity. The model nonetheless occasionally reasons in plain text ending with a `</think>` marker. We strip text before the last marker, count it as *leaked reasoning*, and allow 2048 tokens for code and 256 for tool calls. A reply that reaches the token limit is scored as a failure, because any code it contains may be an intermediate draft.

C. Tool-call and strict-JSON cases

The set contains 60 cases generated from templates with a fixed seed. Thirty are tool calls: each prompt offers five function schemas (weather, currency conversion, timer, table booking, stock price) and requires exactly one call with the correct name and argument values and types; small normalizations are embedded, e.g., "8:30pm" must become `"20:30"`. Thirty are strict-JSON extractions (person, order and event records) requiring exact keys, values and types, including booleans and ISO dates. JSON answers are scored as *correct* if the first JSON object in the reply matches, and separately as *clean* if the reply contains only JSON; this rule was fixed on Q8_0 before any other file was tested. Server-side grammar constraints are deliberately not used, since they would hide formatting degradation. The 20 screen cases comprise two per tool template and ten JSON cases.

D. Code screen selection

The 15 screen problems were selected from BF16 results only, before any quantized file was evaluated: among the 114 problems BF16 solved on HumanEval+ without truncation or leaked reasoning, we split problems into three equal bands by the number of non-empty lines in the reference solution and drew five per band with a fixed seed. Solutions are extracted from the reply's code block, with the prompt's imports restored when the model returns a complete function.

E. Thresholds

Table I shows the baselines. BF16 and the near-lossless Q8_0 disagree on five HumanEval+ problems, giving a paired standard error of about 1.4 percentage points. We call a file **functional** if it solves at least 110 of 164 problems (BF16 minus five points, about 3.7 standard errors) and at least 57 of 60 tool cases; **non-functional** if it solves fewer than half of BF16's code score (< 59) or fewer than 30 tool cases;

TABLE I
BASELINES ON THE FULL BENCHMARKS (GREEDY DECODING).

File	HumanEval+	Tool/JSON	Screen
BF16 (reference)	118/164 (72.0%)	60/60	15/15, 20/20
Q8_0	119/164 (72.6%)	60/60	15/15, 20/20
Per-problem disagreements: 5 (2 BF16-only, 3 Q8_0-only); tool: 0			

TABLE II
CENSUS POPULATION AND SCREEN VERDICTS.

Uploader	Files	imatrix in metadata	Pass	Suspect
Official (LiquidAI)	8	0	7	1
bartowski	22	20	13	9
prithivMLmods	9	0	7	2
AtomicChat	6	5	5	1
Total	45	25	32	13

and **degraded** otherwise. On the screen, a file is **suspect** if it solves fewer than 13 of 15 code problems or fewer than 18 of 20 tool cases.

F. Packaging as a GitHub Action

A composite GitHub Action builds llama.cpp at a pinned release (cached across runs, compiled for generic CPUs), downloads the file from Hugging Face, runs the gate on CPU, writes a results table to the job summary and fails the job on a failing verdict. On a harness error, the summary shows the tester's last output lines instead of a verdict.

IV. EXPERIMENTAL SETUP

A. Artifacts

We listed every `.gguf` file in four repositories through the Hugging Face API, recording size, SHA-256 and repository revision (Table II): the official `LiquidAI/LFM3-1.7B-GGUF` (8 files, including a quantization-aware-distilled `QAD-Q4_0` and an `NVFP4` file converted from a separate checkpoint), `bartowski` (22 files, imatrix at every level), `prithivMLmods` (9 files) and `AtomicChat` (6 "Atomic-Dynamic" files with role-specific per-tensor precision). bartowski's imatrix file was excluded as a non-model. Fine-tuned derivatives were excluded because they are different models.

B. Procedure

Census. Each file was downloaded at the inventoried revision, its hash verified (45/45 matched), screened, recorded and deleted; the loop was resumable. **Confirmation.** All 13 suspects and a random sample of 10 passers (fixed seed; the official BF16 and Q8_0 excluded because their full results were already known) were run on the full benchmarks. The sample was committed to version control before any result was produced. **KL.** BF16 logits were saved once with `llama-perplexity` over WikiText-2 (test split; 100 chunks of 512 tokens). Each file was compared against them, yielding mean KL, 99th-percentile KL and top-1 agreement.

TABLE III
FILES WITH FULL-BENCHMARK VERDICTS, SORTED BY BITS PER WEIGHT (BPW). CODE: HUMANEVAL+ SOLVED OF 164; TOOL: OF 60. SCREEN: P = PASS, S = SUSPECT. VERDICT: F = FUNCTIONAL, D = DEGRADED, N = NON-FUNCTIONAL.

Uploader	Quant	bpw	Code	Tool	KL	Scr.	Ver.
bartowski	IQ2_M	3.01	18	56	0.599	S	N
bartowski	Q2_K	3.24	42	59	0.726	S	N
bartowski	IQ3_XXS	3.61	68	57	0.295	S	D
bartowski	Q3_K_S	3.75	70	60	0.374	S	D
bartowski	IQ3_XS	3.76	85	60	0.260	S	D
bartowski	Q3_K_M	3.92	96	60	0.237	S	D
bartowski	Q3_K_L	4.10	96	60	0.202	S	D
prithivMLmods	Q3_K_M	4.23	74	60	0.348	S	D
bartowski	IQ3_M	4.28	104	51	0.177	S	D
prithivMLmods	Q3_K_L	4.47	77	60	0.310	S	D
bartowski	IQ4_XS	4.61	111	60	0.121	S	F
official	QAD-Q4_0	4.85	116	60	0.264	P	F
AtomicChat	AD-IQ4_XS	5.05	113	60	0.102	S	F
AtomicChat	AD-Q4_K_M	5.24	116	60	0.064	P	F
bartowski	Q4_K_L	5.50	113	60	0.062	P	F
official	Q5_K_M	5.82	108	60	0.040	P	D
bartowski	Q5_K_S	5.86	110	60	0.026	P	F
official	NVFP4†	5.92	99	60	0.097	S	D
AtomicChat	AD-Q5_K_M	6.06	120	60	0.023	P	F
bartowski	Q5_K_M	6.25	113	60	0.018	P	F
AtomicChat	AD-Q6_K	7.34	110	60	0.006	P	F
AtomicChat	Q8_0	8.54	116	60	0.002	P	F
official	Q8_0	8.54	119	60	0.002	P	F
bartowski	BF16	16.03	118	60	0.000	P	F
official	BF16	16.03	118	60	0.000	P	F

†Measured on llama.cpp b12128 (Section V-G).

Controlled rebuild. From the official BF16 file we computed an imatrix on a WikiText-2 *training* slice (200 chunks of 512 tokens; disjoint from the KL text) and quantized `Q2_K`, `Q3_K_S` and `Q3_K_M` three ways: without imatrix, with our imatrix, and with bartowski's published imatrix (486 chunks of his general calibration set). For every quantization type, all three files have identical per-tensor types (verified tensor by tensor) and differ in size only by 256 bytes of metadata.

C. Environment

Census and confirmation ran on an Apple M5 MacBook Air (Metal backend). Quantization and KL used llama.cpp b12128 built from source. Owing to a harness defect described in Section VI, generation in the census, confirmation and rebuild used llama.cpp b11960, a release that supports LFM3. Rerunning four files on b12128 reproduced their screen scores exactly, and the NVFP4 file, which requires b12088 or newer, was re-measured on b12128. CI runs use b12128 on GitHub's Linux CPU runners.

V. RESULTS

A. Census and confirmation

Table III lists all 25 files with full-benchmark results (the 23 confirmed files, the NVFP4 retest and the two baselines). Thirteen were functional, ten degraded and two non-functional.

B. Screen accuracy

Table IV compares screen and full verdicts for the 23 confirmed files. Among the 10 sampled passers, one was

TABLE IV
SCREEN VERDICT VS. FULL-BENCHMARK VERDICT (23 CONFIRMED FILES).

	Full benchmark		
Screen	Functional	Degraded or worse	Total
Pass	9	1	10 (sample)
Suspect	2	11	13 (all)

degraded on the full benchmark: the official `Q5_K_M`, at 108 versus the 110 threshold (miss rate 1/10, Wilson 95% CI 1.8%–40.4%). Among the 13 suspects, two were functional: bartowski's and AtomicChat's `IQ4_XS` files, which scored 12/15 on the screen (one problem below its threshold) and 111 and 113 on the full benchmark (false-alarm rate 2/13, CI 4.3%–42.2%). All three errors lie within one problem of a threshold, and no non-functional file passed the screen. The interval on the miss rate is wide: ten sampled passers cannot exclude a meaningful miss rate.

The screen took a median of 264 s per file (including download and KL) against 1862 s for the full benchmarks, about 7× faster. With `--fail-fast`, our non-imatrix `Q2_K` was rejected in 30 s. On GitHub's free CPU runner, the Q8_0 screen took 441 s.

C. H1: where the model breaks

Fig. 1 plots code scores against bits per weight. All ten fully benchmarked files below 4.5 bits per weight were degraded or non-functional; the two smallest (`IQ2_M`, 3.01 bpw; `Q2_K`, 3.24 bpw) solved 18 and 42 problems. The smallest functional file was bartowski's `IQ4_XS` (4.61 bpw, 0.997 GB, 111 solved). One file in this range, AtomicChat's `AD-IQ3_S` (4.26 bpw), passed the screen 15/15 but was not in the confirmation sample. Above 4.5 bpw, two files fell below the functional threshold: the official `Q5_K_M` (108) and NVFP4 (99). H1 is therefore *partially supported*: failures concentrate at low bit widths, but the boundary for this 1.7B model lies near 4.5 bits per weight rather than 3, and file size alone does not guarantee quality above it.

D. H2: same name, different file

At every quantization name published by both bartowski (imatrix) and a plain uploader, bartowski's file drifted less from BF16, with mean KL 17.7% (`Q4_K_S`) to 55.1% (`Q5_K_M`) lower (Fig. 2). On the full benchmark, bartowski's `Q3_K_M` and `Q3_K_L` solved 96 problems each, against 74 and 77 for prithivMLmods' files, and bartowski's `Q5_K_M` solved 113 against 108 for the official file. The official and prithivMLmods files had identical KL to six decimals at `Q4_K_M`, `Q5_K_M`, `Q6_K` and `Q8_0`, and prithivMLmods' `Q3_K_M` has the same tensor-type histogram and size as a default no-imatrix build.

These census pairs confound imatrix with the per-tensor precision mix (bartowski's `Q3_K_M` is 0.847 GB versus 0.915 GB), so the controlled rebuild isolates calibration (Table V, Fig. 3). At 2 bits, either imatrix raised `Q2_K` from

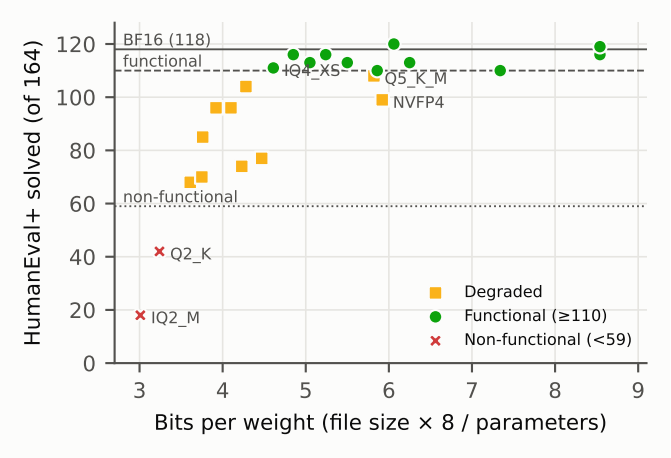


Fig. 1. HumanEval+ problems solved vs. bits per weight for the 23 files with full results below 9 bpw (BF16 at 16 bpw omitted). Horizontal lines: BF16 score, functional and non-functional thresholds.

TABLE V
CONTROLLED REBUILD: IDENTICAL PER-TENSOR TYPES, ONLY THE IMPORTANCE MATRIX DIFFERS. CODE OF 164 / TOOL OF 60, AND MEAN KL.

Quant	No imatrix	WikiText-2 (200 ch.)	bartowski (486 ch.)
Q2_K	1 / 36	55 / 60	51 / 51
	KL 1.101	KL 0.839	KL 0.908
Q3_K_S	96 / 60	83 / 60	78 / 60
	KL 0.515	KL 0.395	KL 0.434
Q3_K_M	76 / 60	86 / 60	104 / 60
	KL 0.349	KL 0.258	KL 0.261

1 to 51–55 solved problems and restored tool calling (36 to 51–60 of 60). At 3 bits the picture changes. With bartowski's calibration, `Q3_K_M` solved 104 problems (+28 over no imatrix); with our WikiText-only calibration it solved 86 (+10, not significant); at `Q3_K_S` both imatrix arms scored *below* the no-imatrix build (83 and 78 vs. 96).

Table VI reports exact McNemar tests on per-problem outcomes. With 18 comparisons (Bonferroni threshold $\alpha = 0.0028$), imatrix versus none at `Q2_K` (code and tools, both calibrations) and bartowski's calibration versus none at `Q3_K_M` ($p = 0.0002$) remain significant; bartowski versus WikiText calibration at `Q3_K_M` ($p = 0.0039$) narrowly misses the corrected threshold, and the `Q3_K_S` reversal is suggestive only. H2 is *supported with a qualification*: the name does not determine the file, imatrix is decisive at 2 bits, and at 3 bits the calibration data matters more than whether calibration is used. Because bartowski's arm differs in both calibration text and amount, we cannot attribute its advantage to one or the other.

E. H3: KL divergence as a screen

Across the 25 files with full results, mean KL correlated strongly with code score (Spearman $\rho = -0.84$; Fig. 4). To test KL as a gate, we searched every cutoff for the one that best separated functional from non-functional or degraded files *on*

TABLE VI
EXACT MCNEMAR TESTS ON PER-PROBLEM OUTCOMES IN THE REBUILD. b: SOLVED ONLY BY ARM A; c: SOLVED ONLY BY ARM B. ALL TOOL COMPARISONS AT 3 BITS WERE 60 VS. 60 ($p = 1$).

Quant	Task	A vs. B	A	B	b/c	p
Q2_K	code	none vs. WikiText	1	55	0/54	$< 10^{-4}$
Q2_K	code	none vs. bartowski	1	51	0/50	$< 10^{-4}$
Q2_K	code	WikiText vs. bartowski	55	51	23/19	0.644
Q2_K	tool	none vs. WikiText	36	60	0/24	$< 10^{-4}$
Q2_K	tool	none vs. bartowski	36	51	4/19	0.0026
Q2_K	tool	WikiText vs. bartowski	60	51	9/0	0.0039
Q3_K_S	code	none vs. WikiText	96	83	28/15	0.066
Q3_K_S	code	none vs. bartowski	96	78	33/15	0.013
Q3_K_S	code	WikiText vs. bartowski	83	78	17/12	0.458
Q3_K_M	code	none vs. WikiText	76	86	21/31	0.212
Q3_K_M	code	none vs. bartowski	76	104	13/41	0.0002
Q3_K_M	code	WikiText vs. bartowski	86	104	9/27	0.0039

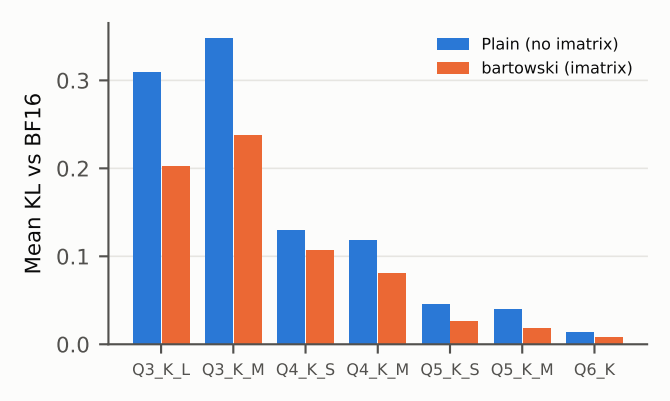


Fig. 2. Mean KL vs. BF16 for census files with the same quantization name: plain build (official or prithivMLmods) vs. bartowski's imatrix build.

these same files, which flatters KL. The best cutoff (0.121) still made three errors (Table VII): it admitted the official `Q5_K_M` (KL 0.040, 108 solved) and NVFP4 (KL 0.097, 99 solved), and rejected `QAD-Q4_0` (KL 0.264, 116 solved). The QAD file was retrained to tolerate 4-bit quantization, so its outputs legitimately diverge from BF16 while its capability is preserved. The task screen also made three errors, but two were false alarms (the safe direction) and all were within one problem of a threshold.

The rebuild provides the clearest case. Our two imatrix builds of `Q3_K_M` have identical tensor types and nearly identical KL (0.258 and 0.261) yet solve 86 and 104 problems ($p = 0.0039$). Across the rebuild, imatrix lowered KL in every arm, including the `Q3_K_S` arms that solved fewer problems. H3 is *supported*: KL is a useful ranking signal and a poor acceptance criterion.

F. H4: tool calling vs. code

Contrary to H4, tool calling was the more robust capability (Fig. 5). In the 25 public files with full results, tool/JSON accuracy never fell below 85% of BF16 (`IQ3_M`: 51/60) and was at least 93% in all other files, while code fell to 15% (`IQ2_M`: 18/118 of BF16's score). Tool calling does

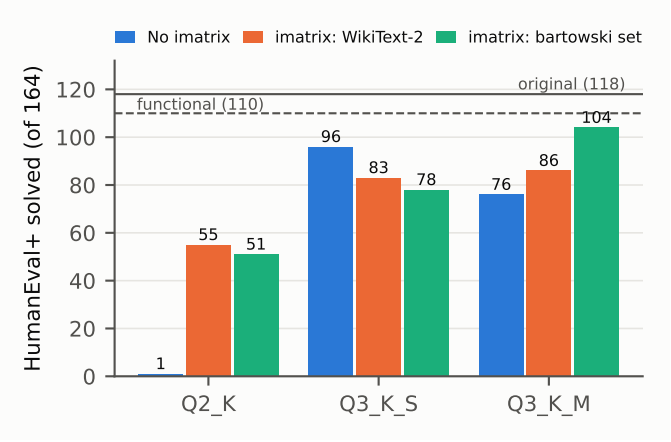


Fig. 3. Controlled rebuild: HumanEval+ problems solved for three calibration arms at three quantization types.

TABLE VII
ERRORS OF THE BEST IN-SAMPLE KL CUTOFF AND OF THE TASK SCREEN.

	Admits a failing file	Rejects a functional file
KL cutoff 0.121	2 (Q5_K_M, NVFP4)	1 (QAD-Q4_0)
Task screen	1 (Q5_K_M)	2 (both IQ4_XS)

break eventually: our no-imatrix `Q2_K` failed 40% of tool cases while solving one code problem. JSON formatting cleanliness degraded earlier than correctness in some files (e.g., `IQ3_XXS`: 60% clean). H4 is *rejected* for this model and task set, with the caveat that the tool cases are easier than HumanEval+ (BF16 scores 100%), so the tool screen has less headroom to register degradation.

G. NVFP4: a runtime incompatibility

The official NVFP4 file keeps feed-forward weights in NVIDIA's 4-bit floating-point format and requires llama.cpp b12088 or newer. On b11960 it loaded without error and generated only repeated "G" characters, scoring 0/15 and 0/20; the gate rejected it. A load-only check would have accepted it, and its KL (measured with b12128) was 0.097, typical of functional 4-bit files. On b12128 it worked but was degraded: 12/15 on the screen and 99/164 on the full benchmark with 60/60 tool cases. Its metadata, tokenizer and chat template were identical to the official Q8_0. This case illustrates a failure category orthogonal to weight quality: a correct file paired with an incompatible runtime.

H. Secondary observations

Lower bit widths were associated with more leaked reasoning (Spearman $\rho = -0.54$ between bits per weight and leaked replies on the code screen) and more truncated replies ($\rho = -0.73$; 44 files, NVFP4 excluded). On the full benchmark, our no-imatrix `Q2_K` reached the token limit on 162 of 164 problems. Degraded files thus fail partly by rambling, which also makes them slow to evaluate and makes fail-fast valuable. AtomicChat publishes its own KL and top-1 agreement for

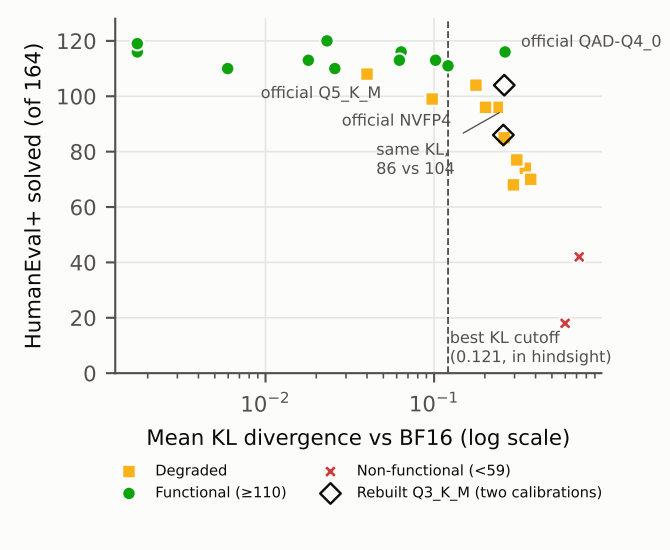


Fig. 4. Mean KL vs. BF16 against HumanEval+ problems solved. Diamonds: the two imatrix-calibrated rebuilds of Q3_K_M, nearly identical in KL but 18 problems apart.

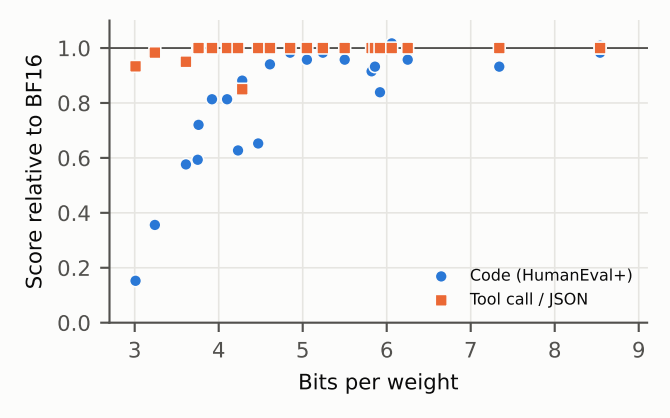


Fig. 5. Code and tool/JSON scores relative to BF16 vs. bits per weight.

its files on a 30-language set; our English-only measurements rank its files identically and are uniformly lower.

VI. Harness Defects and Safeguards

Four defects in our pipeline each produced plausible but wrong numbers. We report them because each is likely to recur in any similar harness.

The fourth defect deserves emphasis. Reports recorded the version of the pinned binary while the server launched a different one, so all generation results were labelled with the wrong version. The discrepancy surfaced only when the GitHub Action ran on a machine without the older binary and the gate exited with code 3. During the NVFP4 diagnosis it also produced a misleading intermediate conclusion: one diagnostic that passed the pinned binary explicitly produced coherent text, while later diagnostics using the default produced garbage, which we initially attributed to a stray server.

TABLE VIII
Harness defects found during the study.

Symptom	Cause	Safeguard
Q8_0 solved 0/5 easy problems	EvalPlus's memory limit (`RLIMIT_AS`) is rejected on macOS; every test crashed and was logged as "timeout"	Disable the memory limit on macOS; abort on sandbox errors or if most tests time out
Code score ∼16 points low	Reasoning leaked into replies despite thinking off; replies truncated at 768 tokens	Strip and count leaked reasoning; 2048-token budget; truncation is a failure
Correct JSON marked wrong	Stray code fence after valid JSON	Separate "correct" and "clean"; rule fixed on Q8_0 first
Results attributed to the wrong runtime	Server launched the first `llama-server` on `PATH` (b11960) instead of the pinned build	Pinned binary path; each report records the binary actually launched; CI on a clean machine

After the fix, four files rerun on b12128 reproduced their screen scores exactly, so the remaining results are unaffected; NVFP4 was the exception. Other checks support determinism: rerunning BF16 reproduced 15/15; screen scores derived from full runs matched real screen runs; and Q8_0 scored identically on Apple Metal and on Linux CPU.

VII. Discussion

For users. The quantization name and file size are weak guides to quality for small models. For LFM3-1.7B, functional files exist from 1.0 GB (bartowski `IQ4_XS`, 111) and the best-scoring file under 1.5 GB was AtomicChat's `AD-Q5_K_M` (1.31 GB, 120); the retrained `QAD-Q4_0` (1.05 GB, 116) shows that training for quantization can beat larger post-training quantizations. Every fully tested file under 1 GB was degraded or worse for coding.

For registries and uploaders. A short task-based screen catches the files that matter at modest cost, and should complement rather than be replaced by KL or perplexity. Recording calibration provenance (data, amount, tool version) matters as much as recording whether an imatrix was used, since at 3 bits two calibrations of identical tensor layout differed by 18 problems. Runtime requirements should be machine-readable, since a file that "loads fine" on an unsupported runtime can be useless.

For tool builders. A gate must be able to report that it failed, distinct from reporting that the file failed, and must record the runtime it actually used.

VIII. Threats to Validity

External validity. We studied one model family and one runtime. Thresholds are calibrated to LFM3-1.7B and must be re-derived for other models. LiquidAI does not position the model for coding, so HumanEval+ measures a capability it is not optimized for; it nonetheless provides an objective, executable signal. **Construct validity.** The 60 tool/JSON cases were written for this study and are easier than HumanEval+

(ceiling at 100%), limiting the tool screen's sensitivity and qualifying the H4 result. Greedy decoding with reasoning disabled differs from the vendor's recommended sampling. **Statistical conclusion validity.** The confirmation sample of 10 passers yields wide intervals. Functional and non-functional thresholds involve judgment, although they are anchored to measured BF16–Q8_0 noise. The KL cutoff analysis is in-sample, which favors KL. **Internal validity.** Generation used llama.cpp b11960 while quantization and KL used b12128; a four-file rerun showed no difference, but we did not rerun every file. The rebuild's bartowski arm differs from ours in both calibration text and quantity. Code execution used process isolation with time limits, not a security sandbox.

IX. Conclusion

Quantized model files can load cleanly and still fail at their tasks, and for a 1.7B-parameter model this happens below about 4.5 bits per weight and occasionally above it. A 35-case, exactly scored functional screen identified every non-functional file in our census in a median of 4.4 minutes, with errors confined to the threshold boundary, whereas KL divergence, though well correlated with quality, admitted degraded files and rejected a functional one. Controlled rebuilds showed that calibration rescues 2-bit files and that, at 3 bits, the calibration data rather than its presence decides the outcome. We release QUANT-GATE as a command-line tool and GitHub Action, together with the census data, so that the same check can run before a file is published or deployed.

Acknowledgment

This work builds directly on Patodiya's "Broken on Arrival" study and its `quantcheck` tool. We thank the maintainers of llama.cpp and EvalPlus, and the uploaders whose files we evaluated; low-bit files are published as size–quality trade-offs, and this study aims to make those trade-offs measurable.

References

[1] A. Patodiya, "Broken on arrival: Silently defective LLM artifacts in public model registries and how to catch them," *arXiv preprint arXiv:2609.05881*, 2026.

[2] G. Gerganov *et al.*, "llama.cpp: LLM inference in C/C++," 2023–2026. [Online]. Available: https://github.com/ggml-org/llama.cpp

[3] Liquid AI, "LFM3-1.7B," model card, Hugging Face, Oct. 2026. [Online]. Available: https://huggingface.co/LiquidAI/LFM3-1.7B

[4] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," *arXiv preprint arXiv:2210.17323*, 2022.

[5] J. Lin *et al.*, "AWQ: Activation-aware weight quantization for LLM compression and acceleration," *arXiv preprint arXiv:2306.00978*, 2023.

[6] T. Dettmers and L. Zettlemoyer, "The case for 4-bit precision: k-bit inference scaling laws," in *Proc. Int. Conf. Machine Learning (ICML)*, 2023.

[7] W.-L. Liu and C.-Y. Tsai, "What you sign is not what you run: The verifiability gap in quantized LLM supply chains," *arXiv preprint arXiv:2610.19498*, 2026.

[8] S. K. Singh, "Saying, not knowing: Aggressively GGUF-quantized small language models still write rare words they can no longer define," *arXiv preprint arXiv:2610.04403*, 2026.

[9] M. Chen *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[10] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[11] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," *arXiv preprint arXiv:1609.07843*, 2016.

[12] S. Kullback and R. A. Leibler, "On information and sufficiency," *Ann. Math. Statist.*, vol. 22, no. 1, pp. 79–86, 1951.

[13] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.

[14] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[15] O. J. Dunn, "Multiple comparisons among means," *J. Amer. Statist. Assoc.*, vol. 56, no. 293, pp. 52–64, 1961.